

Projet Final

Orchestration Intelligente et Scaling Économique d'Architectures ML Orientées-Service

1. Aperçu et Enjeux Métier

Historiquement, les entreprises achetaient des serveurs physiques pour héberger leurs applications. Elles devaient dimensionner leur parc pour le "pire scénario" (le pic annuel), ce qui signifiait que 90% du temps, ces machines coûteuses tournaient à vide.

Dans l'écosystème actuel (AWS, Azure, Google Cloud), on privilégie la **location de ressources à la demande** (Modèle IaaS).

- **Pourquoi ?** Pour bénéficier de l'**élasticité**. L'infrastructure peut grandir ou rétrécir en temps réel selon les besoins exacts de l'application.
- **Le problème :** Si la location offre une flexibilité théorique, elle impose une gestion complexe au quotidien appelée le **Capacity Planning**.

Sans une orchestration intelligente, on tombe dans deux pièges :

- **Over-provisioning (Gaspillage) :** On loue trop de ressources par sécurité. L'application est performante, mais la facture Cloud explose, annulant le bénéfice économique de la location.
- **Under-provisioning (Risque) :** On loue trop peu pour économiser. Lors d'un pic d'utilisateurs, l'application sature, la latence grimpe et le service devient indisponible (Crash/Déni de service).

Ce projet vous place dans le rôle d'un **Architecte Cloud / SRE (Site Reliability Operator)**. Vous devez gérer une application d'IA complexe, fragmentée en **micro-services**, et concevoir un "Orchestrateur Intelligent" (Autoscaler) capable de :

1. **Minimiser les OPEX** (Dépenses opérationnelles / Facture Cloud).
2. **Respecter les SLA** (Service Level Agreements / Garanties de performance).

Vous devrez mobiliser des algorithmes d'IA d'aide à la décision **vus en cours** pour résoudre ce problème complexe.

2. Cadre du Projet

Règles Fondamentales

- **Groupe :** 3 étudiants maximum.
- **Livrables :** Code source, Rapport d'analyse et Présentation orale.
- **Originalité :** Le travail doit refléter votre propre réflexion. Le plagiat est éliminatoire.
- **Analyse > Code :** Dans le cadre de votre formation, votre valeur ajoutée réside surtout dans la compréhension et la justification de vos choix. Un algorithme qui "fonctionne" sans explication ne suffit pas.

Bonus : Jusqu'à **+2 points** pour une **démarche approfondie** : approche originale (ex : Reinforcement Learning), richesse de l'analyse comparative ou visualisation soignée des résultats et argumentation étayée.

3. Description du Système et Contraintes Techniques

3.1 Architecture : Le Pipeline de Micro-Services

L'application cible repose sur une architecture logicielle moderne de type **micro-services**, un paradigme particulièrement adapté pour orchestrer des systèmes complexes tels qu'une chaîne **d'entraînement et d'inférence distribuée**.

Contrairement aux applications monolithiques traditionnelles (bloc unique), le système est ici décomposé en plusieurs composants autonomes. **Chaque composant (service) est spécialisé et effectue une tâche spécifique** du processus global (nettoyage, calcul, validation, etc.).

Ces services sont organisés sous forme d'une chaîne de traitement séquentielle (Pipeline) comme illustré dans la Figure 1.

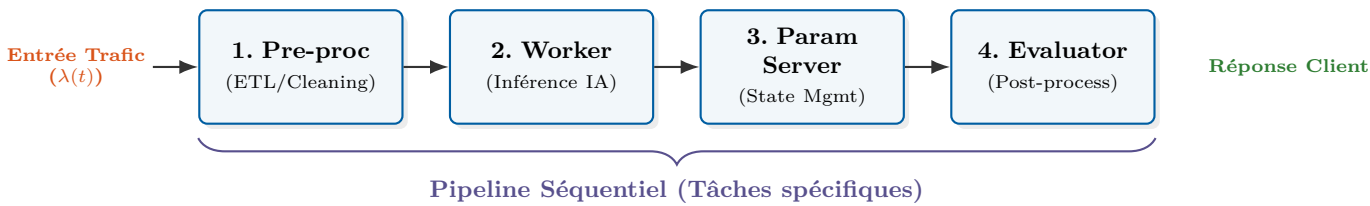


FIGURE 1 – Exemple d'application IA orientée services : Une requête traverse les 4 services spécialisés. Chaque micro-service possède des caractéristiques propres et des exigences spécifiques en matière de ressources (CPU, RAM). **L'ensemble de ces besoins techniques et capacités de traitement est détaillé dans le Tableau 1 ci-dessous.**

TABLE 1 – Spécifications techniques et capacité unitaire des services

ID	Micro-service	vCPU / Instance	RAM / Instance (GiB)	Capacité μ (req/s)
1	Pre-processing	1.0	0.50	1416
2	Model Worker	4.0	2.50	696
3	Parameter Server	2.0	1.50	1005
4	Evaluator	1.0	1.00	1259

3.2 Les Leviers d'Optimisation

1. Analyse du Workload (La Demande)

Le système reçoit un flux de requêtes exogène $\lambda(t)$ qui varie dans le temps (voir `workload.xlsx`). Comme illustré ci-contre, ce trafic est **élastique** : il présente des creux (nuit) et des pics violents (rush hour). Votre système doit être **réactif** : provisionner massivement pendant les pics pour tenir la charge, et dé-provisionner agressivement pendant les creux pour sauver du budget.

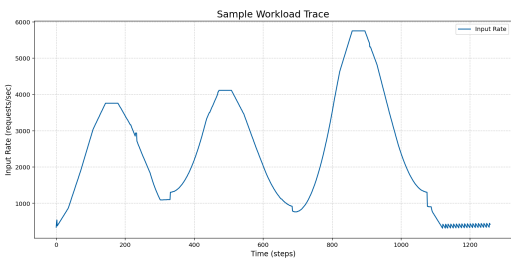


FIGURE 2 – Variabilité temporelle du trafic.

2. Le Scaling Horizontal (La duplication)

Pour absorber une forte demande et éviter la formation de **goulots d'étranglement** (pouvant causer des temps d'attente prohibitifs ou des pertes de requêtes), la stratégie consiste à **dupliquer** le service. Cette méthode implique de déployer en parallèle plusieurs instances identiques du même composant. Chaque réplique conserve ainsi **exactement les mêmes caractéristiques** (CPU, RAM, capacité μ) que le service initial.

La Figure 3 ci-dessous illustre ce mécanisme : une instance unique saturée par la demande est remplacée par plusieurs clones qui se partagent le travail.

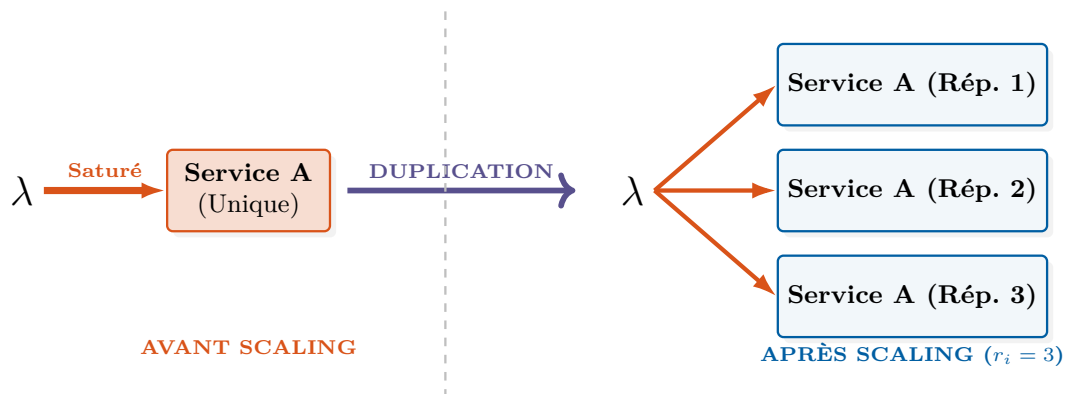


FIGURE 3 – Illustration du passage à l'échelle : duplication des instances.

L'objectif est de déterminer le nombre de répliques r_i pour chaque composant qui garantit une capacité totale de traitement strictement supérieure à la demande (λ), assurant ainsi la stabilité du système.

Règle d'Or de la Stabilité

La capacité totale de vos répliques doit être supérieure à la demande des clients :

$$\text{Nombre de répliques } (r_i) \times \text{Capacité unitaire d'un service } (\mu_i) > \text{Demande Client } (\lambda)$$

3. Allocation des Ressources et Placement (Provisioning)

Une fois le degré de réplication optimal r_i déterminé pour chaque service, l'étape suivante consiste à héberger ces instances sur l'infrastructure Cloud. C'est la phase de **provisioning**.

L'objectif est de sélectionner la combinaison de Machines Virtuelles (VMs) la plus économique capable de supporter la charge de toutes les répliques.

- L'infrastructure est définie par un catalogue hétérogène de **17 types de VMs** (cf. `VM_Catalogue.xlsx`), caractérisés par leur capacité (vCPU, RAM), leur coût horaire et leur **quantité limitée**.
- Ce défi d'optimisation consiste à **minimiser le coût total d'infrastructure**, en respectant scrupuleusement deux contraintes :
 - **Contrainte de capacité** : La somme des ressources (CPU, RAM) consommées par les services hébergés sur une VM ne doit jamais excéder la capacité physique de celle-ci.
 - **Contrainte de disponibilité** : Le nombre de VMs utilisées pour un type donné ne doit jamais dépasser le stock disponible indiqué dans le catalogue.

4. Méthodologie Attendue

1. **Formalisation et Choix d'implémentation** : Formalisez rigoureusement le problème (variables, fonction objectif, contraintes). Expliquez en détail vos décisions structurelles et la raison derrière la formalisation choisie en Python.
2. **Approches de résolution** : Implémentez et présentez **au moins deux algorithmes distincts** parmi ceux **vus** en cours (recherche non-informée, recherche informée, heuristiques, recherche locale, RL, etc.). Détaillez le fonctionnement de chaque méthode et justifiez leur contribution à la solution.
3. **Résultats et Analyse comparative** : Comparez vos solutions via une évaluation approfondie (coût final, temps d'exécution, qualité, etc.). Discutez des forces de votre proposition ainsi que des pistes d'amélioration possibles.
4. **Réflexions finales** : Résumez votre expérience globale sur le projet, les défis techniques rencontrés, la manière dont ils ont été surmontés et vos principaux acquis.

5. Niveaux de difficulté et Barème

Choisissez le niveau de complexité que vous souhaitez traiter. Votre note maximale en dépendra.

Niveau 1 : Le cas statique (Note max : 15/20)

Idéal pour commencer. On se place dans le "pire cas possible".

- **Scénario** : Considérez uniquement la valeur **maximale** de la demande présente dans le fichier `workload.xlsx` (le pic de trafic).
- **Objectif** : Déterminer la meilleure configuration (nombre de répliques r_i , ainsi que le type et le nombre de VMs) qui supporte ce pic **au coût le plus bas**.
- **Étude de Scalabilité** : Analyser le comportement de votre approche de résolution lors du passage à l'échelle : que se passe-t-il si l'on doit gérer 2, 4, 8 ou 10 applications simultanées ?

Niveau 2 : Le cas dynamique (Note max : 17/20)

Approche plus réaliste. On prend en compte les variations de la demande.

- **Scénario** : Considérez **l'intégralité de la trace de trafic** du fichier `workload.xlsx` (la demande varie chaque seconde).
- **Objectif** : Déterminer une **configuration robuste** unique, issue de l'analyse de la trace complète.
- **Démarche de réflexion attendue** :
 1. Calculez la configuration optimale (Nombre de répliques + VMs) pour **chaque instant** t de la trace.
 2. Analysez la fréquence des configurations obtenues pour identifier celle qui offre le meilleur compromis (ex : la configuration qui permet d'absorber 95% du trafic ou la plus fréquente).
- **Analyse Critique** : Comparez cette configuration "robuste" avec celle du Niveau 1 (basée sur le pic absolu). Discutez du gain économique obtenu et du risque accepté (les requêtes non couvertes).
- **Étude de Scalabilité** : Analyser le comportement de vos algorithmes lors du passage à l'échelle (gestion de 2, 4, 8 ou 10 applications simultanées).

Niveau 3 : L'Expertise (Excellence)

Complexe. On ajoute la gestion des temps de réponse.

- **Scénario** : Trafic variable + Contrainte de latence (SLA).
- **Nouveauté** : Estimer le temps de réponse via **l'approximation par le taux d'occupation** (saturation).
Plus le système est chargé, plus il est lent. Utilisez cette formule simplifiée :

$$\text{Temps de Réponse} \approx \frac{1}{(r_i \times \mu_i) - \lambda(t)}$$

On voit ici que si le nombre de répliques r_i augmente, le dénominateur grandit et le temps de réponse diminue.

- **Votre but** : Optimiser deux objectifs contradictoires :
 1. Minimiser le coût d'infrastructure (Argent).
 2. Minimiser le temps de réponse calculé (Qualité).
- **Analyse Critique** : Analysez le **compromis Coût / Performance**. Montrez comment le budget évolue lorsque l'on exige une réponse plus rapide. Identifiez le moment où le gain de vitesse devient trop faible pour justifier l'explosion du prix. Enfin, **discutez de la meilleure configuration à adopter selon le contexte** : laquelle choisiriez-vous pour une startup (budget limité) et laquelle pour une banque (performance critique) ?

6. Calendrier et Livrables

Jalon	Date Limite	Contenu et Spécifications
Présentation Orale	21 janv. 2026	Soutenance (15 min) Explication synthétique de votre solution : . Description et formalisation du problème. . Justification des choix d'implémentation. . Description et justification des deux approches. . Discussion des résultats et enseignements clés.
Rendu Final	23 janv. 2026	Archive ZIP (Notebook + Slides) Le Notebook doit obligatoirement inclure : 1. Code et Visualisation : Code source complet (Formalisme, Algorithmes) et graphiques pertinents (Coûts, Latence, Charge). 2. Choix d'implémentation : Explication des décisions structurelles et du formalisme mathématique retenu. 3. Approches de résolution : Description détaillée des stratégies algorithmiques comparées et de leur logique. 4. Analyse des Résultats : Évaluation chiffrée (Temps, KPI), analyse critique et <i>justification des comportements observés</i>. 5. Réflexions finales : Bilan de l'expérience, défis techniques surmontés et compétences acquises.